iteam | INSTITUTO TECNOLÓGICO EDUCACIONAL DA AMAZÔNIA

AULA 10
PROGRAMAÇÃO ORIENTADA A OBJETOS

Programação em Python • Parte 2 de 2
Curso de Capacitação em Desenvolvimento Full Stack
Tema: Programação Orientada a Objetos • 10 horas
Professor: Msc. Hygo Sousa de Oliveira

(95) 99173-5676 / (92) 99171-0391
www.iteam.org.br
Av. Glaycon de Paiva nº 1820,
Mecejana
CEP: 69.304-560
Boa Vista/RR
CNPJ: 48.653.884/0001-00

Roteiro da aula

O que vamos ver nesta aula

1. Revisão: o que o código imprime?
2. Métodos especiais: str, repr, eq, add e del
3. Composição, agregação e classes abstratas
4. Boas práticas e revisão geral do curso
5. Projetos finais: Biblioteca Digital e Salão ITEAM

Revisão: o que o código imprime?

Preveja a saída antes de executar

QUESTÃO 1

```
class A:
    x = 1

    def __init__(self):
        self.y = 2

a, b = A(), A()
a.x = 10
print(a.x, b.x, A.x)
```

QUESTÃO 2

```
class Pai:
    def oi(self):
        return "Pai"

class Filho(Pai):
    def oi(self):
        return super().oi() + "!"

print(Filho().oi())
```

EM SALA Peça que a turma escreva a previsão no caderno, compare com os colegas e só então execute.

Métodos especiais (dunder)

Nomes com dois underscores que o Python chama por você

Método	Quando o Python o chama	Exemplo
`__init__`	Ao criar o objeto (construtor)	`Carro("Uno")`
`__str__`	Texto para pessoas	`print(obj)`
`__repr__`	Representação para programadores	`repr(obj), listas`
`__eq__`	Comparação de igualdade	`a == b`
`__add__`	Sobrecarga do operador +	`a + b`
`__len__`	Tamanho do objeto	`len(obj)`
`__del__`	Destruição do objeto (destrutor)	`del obj`

__str__ e __repr__

Controlando como o objeto aparece na tela

str_repr.py

```python
class Ponto:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __str__(self):
        return f"({self.x}, {self.y})"

    def __repr__(self):
        return f"Ponto({self.x}, {self.y})"

p = Ponto(2, 3)
print(p)
print([p])
```

SAÍDA

```
(2, 3)
[Ponto(2, 3)]
```

NOTA `print(p)` usa `__str__`. Dentro de listas e no depurador, o Python usa `__repr__`.

__eq__ e sobrecarga de operadores

Definindo o que == e + significam para a sua classe

vetor.py

```python
class Vetor:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __eq__(self, o):
        return self.x == o.x and self.y == o.y

    def __add__(self, o):
        return Vetor(self.x + o.x, self.y + o.y)
```

uso

```python
a = Vetor(1, 2)
b = Vetor(1, 2)
print(a == b, a is b)
soma = a + b
print(soma.x, soma.y)
```

SAÍDA

```
True False
2 4
```

ATENÇÃO Sem `__eq__`, `a == b` compara identidade e daria `False`. `is` sempre compara identidade.

Construtor e destrutor

__init__ cria o objeto e __del__ roda quando ele é destruído

destrutor.py

```python
class Arquivo:
    def __init__(self, nome):
        self.nome = nome
        print(f"Abrindo {nome}")

    def __del__(self):
        print(f"Fechando {self.nome}")

a = Arquivo("notas.txt")
print("Usando o arquivo")
del a
print("Fim")
```

SAÍDA

```
Abrindo notas.txt
Usando o arquivo
Fechando notas.txt
Fim
```

DICA `__del__` roda quando o último vínculo com o objeto some. Para liberar recursos, prefira `with`.

Composição e agregação

O objeto "tem um" outro objeto: quem cria e quem recebe?

garagem.py

```python
class Motor:
    def __init__(self, cv):
        self.cv = cv

class Carro:
    def __init__(self, modelo, cv):
        self.modelo = modelo
        self.motor = Motor(cv)
        self.motorista = None

    def entrar(self, pessoa):
        self.motorista = pessoa
```

uso

```python
class Pessoa:
    def __init__(self, nome):
        self.nome = nome

ana = Pessoa("Ana")
c = Carro("Uno", 75)
c.entrar(ana)
print(c.motor.cv, c.motorista.nome)
del c
print(ana.nome)
```

SAÍDA

```
75 Ana
Ana
```

NOTA **Composição**: `Carro` cria o `Motor`. **Agregação**: recebe uma `Pessoa`.

Classes abstratas

Definir um contrato que as filhas são obrigadas a cumprir

pagamento.py

```python
from abc import ABC, abstractmethod

class Pagamento(ABC):
    @abstractmethod
    def pagar(self, valor):
        ...

class Pix(Pagamento):
    def pagar(self, valor):
        return f"Pix de R$ {valor:.2f}"
```

uso

```python
print(Pix().pagar(50))
try:
    Pagamento()
except TypeError:
    print("Não instancia")
```

SAÍDA

```
Pix de R$ 50.00
Não instancia
```

NOTA `@abstractmethod` obriga cada filha a implementar `pagar`. A classe abstrata em si não pode ser instanciada.

Exemplo guiado: pedido com itens

Desafio: o pedido agrupa itens, soma o total e se apresenta com __str__

loja.py

```python
class Item:
    def __init__(self, nome, valor):
        self.nome = nome
        self.valor = valor

class Pedido:
    def __init__(self):
        self.itens = []

    def incluir(self, item):
        self.itens.append(item)

    def total(self):
        return sum(i.valor for i in self.itens)

    def __str__(self):
        return f"Pedido: R$ {self.total():.2f}"
```

uso

```python
p = Pedido()
cafe = Item("Café", 6.5)
p.incluir(cafe)
p.incluir(Item("Pão", 4))
print(p)
```

SAÍDA

```
Pedido: R$ 10.50
```

Projeto Salão ITEAM: como atacar

14 bugs intencionais espalhados pelos conceitos de POO

1. Ler o diagrama
heranças, interface abstrata, composição e agregação

2. Executar
rode o main.py e leia o erro

3. Localizar
siga o traceback até a classe e compare com o diagrama

4. Corrigir
e justificar cada correção

DICA Meta: o programa roda sem erros e o relatório financeiro é calculado corretamente. Corrija um bug por vez e rode de novo.

Boas práticas e revisão do curso

O que levar do curso para os próximos projetos

Nomes claros

Variáveis, funções e classes que explicam o que fazem.

Funções curtas

Cada função ou método faz uma coisa só.

Docstrings

Documente o que a função recebe e devolve.

Tratar erros

Valide entradas e capture exceções específicas.

Composição

Prefira compor objetos a criar heranças profundas.

Git

Commits pequenos e mensagens que explicam o porquê.

Atividades da Aula 10

Módulo 5: projeto integrador e entrega final

- **Exercício 05 – Projeto integrador**
  Biblioteca Digital, combinando todos os conceitos do módulo
- **Projeto Salão ITEAM (N3)**
  Localizar, corrigir e justificar os 14 bugs seguindo o diagrama de classes
- **Entrega final das três avaliações**
  Registry (N1), Sistema de Notas (N2) e Salão ITEAM (N3), via GitHub

GIT Enunciados, instruções de execução e de entrega:
github.com/Hygo/python-fullstack-iteam-2sem

Obrigado(a)!

Encerramento do curso: bom código e bons projetos!
ITEAM – Instituto Tecnológico Educacional da Amazônia